

Ada95 Question Booklet

OBJEKTUM LIMITED

Airport House, Purley Way, Croydon, Surrey, CR0 0XZ, United Kingdom

t: +44 (0)845 199 9932 e: info@objektum.com w: www.objektum.com

Contents

Table of Contents

Contents 1
An Introduction to Ada 2
Simple Numeric Types 3
Enumeration Types 4
Real Types 5
Record Types 6
Simple Array Types 7
Program Control Flow 8
Subprogram & Operators 9
Packages 10
Private Types 11
Hierarchical Libraries 12
Ada83 Generic Programming 13
Ada95 Generic Programming 14
Ada83 Access Types 15
Ada95 Access Types 16
Object-Oriented Programming 17
Controlled Types 18
Ada83 Exceptions 19
Ada95 Exceptions 20
Ada83 Tasking 21
Ada95 Tasking 22
Interfacing 23
Variant Record Types 24
Unconstrained Array Types 25
Discriminated Record Types 26
Ada95 Predefined Library 27
Ada 2005 – Interfaces 28
Ada 2005 – Access Types 29
Ada 2005 – Synchronized Interface 30
Systems Programming Annex 31
Real-Time Systems Annex 32
Numerics Annex 33
Safety & Security Annex 34

An Introduction to Ada

1. Write a main program to print your name and address on the screen.

2. Change the name of your main procedure to one of the reserved words, try recompiling your program.

3. Now print your age on the screen using Integer_IO. Try printing in base 2, 8, 10 and 16 and with varying widths.

4. Print the value of PI on the screen.

Simple Numeric Types

1. Declare two integers to hold a rectangles height and width, Give these hard coded values of 3 and 4. Calculate the area of the rectangle and display the result on the screen.

2. Remove the hard coded values and prompt the user to enter the height and width.

3. Write a program to display the values returned when the attributes 'First and 'Last are applied to Integer, Natural and Positive types.

4. Declare two objects of type Integer and give them different initial values. Print the result on the screen, when:

    - The two objects are added together
    - One is subtracted from the other
    - The two objects are multiplied together
    - One is divided by the other

5. Write a program that contains a type whose range is the maximum number of values (signed) that can fit into 8 bits. Print the minimum and maximum values of the type using attributes.

6. Declare a type to represent Time and one to represent Distance. Ask the user to input the distance covered and the Time taken. Display the average speed travelled. Note: Ensure types are defined for Speed, Time and Distance (type conversion will be required).

7. Based on the exercise above, display a message if the speed travelled exceeds MACH1. Note: declare a constant to represent MACH1 (741mph).

8. Declare a type to represent people's ages. Declare three subtypes, one to represent the ages of children, one for adults and one for pensioners. Declare an object of each and assign them initial values. Now,

    - Display the age of each person on the screen
    - Assign the age of the adult to the pensioner object
    - Assign the age of the adult to the child (what happened?)

Enumeration Types

1. Declare an enumeration type ‘Aircraft_Type’ that contains the following values (Boeing 747, Tri Star, Harrier, EFA, Hawk, Tornado), in that order. Declare individual objects to hold the values returned when:

    - The attribute ‘Pos is applied to Tornado
    - The attribute ’Succ is applied to EFA
    - The attribute ’Pred is applied to HAWK
    - The attribute ’Pred is applied to Boeing 747

Display the value of the objects on the screen.

2. Print the number of enumerates in the set (No hard coding). Add another enumerate to the set and check the number of elements automatically change.

3. Define two subtypes, one to represent military aircraft and one for commercial aircraft.



Question: Can you spot any potential problem?

4. Change the subtypes to separate types and include C130 in both sets. Declare an object of each enumeration type and set their value to C130

5. Ask the user to input two numbers. Declare an object of type Boolean and sets its value according to whether the first number entered was greater than the second. Print the value of the Boolean object.

6. Write a program to print out the following: True **‘and’** False, False **‘or’** True & True **‘xor’** True.

Real Types

1. Write a main program that declares a floating point type with 4 digits of precision and a fixed point type with an accuracy of 0.1.

   Define three constant values of PI, one with the predefined type Float and the others using the declared types.

   Print all three values of PI to the terminal.

   Note: You will need to use Float_IO and Fixed_IO.

Write a program to calculate the area of a circle whose radius is entered at the terminal (as a float). The result should be printed on the screen.

Record Types

1. Create a record to represent a screen position:
    - Row (1..24)
    - Column (1..80)

    Declare an object to represent a screen position, assign it some values and Display object values on the screen.

2. Create a record type to represent softkey details for a screen; the details should include:
    - Screen Position
    - The Legend (1..4)
    - Orientation (Horizontal / Vertical)

    Declare an object of the new record type and assign it default values. Display object values on the screen.

3. Display the number of bits the records occupy.



Note: Work out the expected result before running your program.

Simple Array Types

1. Assuming a system contained five softkeys, create a structure representing the details for each key. The details should include the legend and the legend position (row / column). Create a constant to hold details for the softkeys, populate it with example details. Print all the details for each key on the screen.

   Note: Use records from record types exercises

2. Using attributes, display the length of the array and the value of the first and last indexes. Display the number of bits the array occupies.

   Note: Work out the expected result before running your program

3. Print the first character (**ONLY**) of each keys legend.

4. Assuming that the system has two pages. Create an array of arrays to represent all of the softkeys.

Weapon	
Key 1	Menu
Key 2	Maint
Key 3	Target
Key 4	Fire
Key 5	Exit

Nav	
Key 1	GPS
Key 2	Adjust
Key 3	Nav
Key 4	Alt
Key 5	Posn

Print all the details for Weapon Page Key 2 and Nav Page Key 5.

Program Control Flow

1. Write a program to print all the values in the type character.

2. Write a program that prompts for the number of items to be entered, and then loops to take that number of numbers as input. Calculate the average and display the result.

3. Write a program that prompts the user to enter a year and then classifies whether or not it is a leap year. It is a leap year if:

   - It is divisible by 4, and
   - It is either not divisible by 100, or it is divisible by 400.

Leap Year	Non Leap Years
1996	1900
1984	1967
2000	1998

4. Using an indefinite loop, write a program to enter a character at the terminal. Using an elsif construct print “digit” if the number is in the range 0.. 9, “Upper Case” if an upper case character is entered, “Lower Case” if a lower case character is entered. Any other input shall result in “Invalid” being printed.

    ***Note: Use subtypes and membership tests to determine the range***

5. Modify the above program to use the case construct.

6. Modify the program to exit the loop when the “Invalid” branch is taken.

7. Modify the above program to use the while construct. The exit condition should remain the same.

8. Write a program that contains an enumeration type, which represents the months of the year. Loop backwards through the enumeration type printing each enumerate.

Modify Simple Array Types exercise question 4 to loop through the arrays and print all the information to the screen.

Subprogram & Operators

1. Write a procedure that contains an internal function that returns True or False according to whether an integer parameter is negative. The main program should contain a loop that reads an integer, passes it to the internal operation and prints the result. The loop should exit if a value of zero is entered.

2. Write a procedure that contains an internal function that takes in a parameter of an enumeration type and returns its position within the enumeration list. Define an enumeration type and loop through it passing the value into the function. Print the result returned by the function.

3. Write a procedure that contains an internal procedure "SWAP" to interchange two parameters of type integer. The main program should contain a loop that reads two integers, passes them to the internal operation and prints the result. The loop should exit if two values of zero are entered.

4. Write a procedure that contains an internal procedure to display a string. The procedure should have a parameter defaulted to true which is used to determine whether the string is printed either forward or backwards. The main program should call the internal procedure twice, once without the defaulted parameter missing and once with it set false.

5. Write a program that contains an internal function that appends two strings together. The strings should be passed as parameters to the function. The main procedure should declare two objects of type string and assign them initial values. Declare a third string and assign it the result returned from the function call (the first two objects should be passed to the function as parameters).

6. Rename the function in the exercise above to "+"

7. Write a procedure that contains an internal function "<" that takes two parameters of type string and returns True if there are less characters on the left hand side. Declare two strings of different sizes, call the function and print the result.

Packages

1. Write a package ‘Stack’ that allows integer values to be:

Pushed	- Stored on the Stack
Popped	- Read (and consumed) from the stack
Size_Of	- Return the number of elements on the stack

 ***Question: What happens if you keep pushing and never pop?***

Private Types

1. Modify the stack package to allow multiple stacks to be declared by the users of the stack package, i.e. The package declares a type for the stack, and objects of that type must be passed to the operations push & pop etc.

Ensure that the stack implementation details are hidden from the user.

The main program should declare two stacks and push the same values onto each. The two stacks should be compared and the message "**Are the same**" should be printed if the comparison returns True.

The main program should then Pop all the values from the stack and display the returned values.

2. Modify the Stack specification so that it is impossible for anyone to copy the stack automatically. The stack package however should provide the facility to allow stacks to be copied.

Hierarchical Libraries

1. Without modifying the original stack specification, extend the stack package to provide two extra facilities:

    a. Allow a user to determine how many items are currently stored on the stack.
    b. Allow a user to display the stack contents.

The main program should use the extended facilities to display the number of elements on each stack and the actual stack contents.

Ada83 Generic Programming

1. Write a the generic procedure swap and an instantiation to test it

2. Modify the stack package to allow any integer data type to be pushed and popped.

The stack package should include:

Push	- Stored a value in the buffer
Pop	- Reads (and consumes) a value from the buffer
Display	- Displays the current stack contents

The generic parameters should include:

Item_Type	- The required type to be stored
Stack_Size	- The required stack size

Instantiate the stack to store up to 20 integer values
Push 10 values in the stack
Check the display operation works

3. Time permitting, modify the stack to allow any data type to be stored.



Note: The item Type should be made private and extra generic parameters are required to enable the display operation to work.

Ada95 Generic Programming

1. Modify the generic stack (from Ada83 Generics, exercise 1) so that the buffer index type (which is circular) is passed to the generic and so that any data type can be stored in the buffer, ie. Make the type private. The Display operation will no longer work.

   To allow the buffer contents to still be displayed, modify the generic specification to include two further formal parameters:

   - Function 'To_Integer (which converts a buffered item to an integer)
   - Package 'Display_Utilities (instantiation of Integer_IO)

When the buffer contents are to be displayed, each element in the buffer should be passed in to the function which converts the element to an integer. The result can then be passed directly into Display_Utilities.Put.



Note: Ensure the actual parameter type for Display_Utilities is always integer.

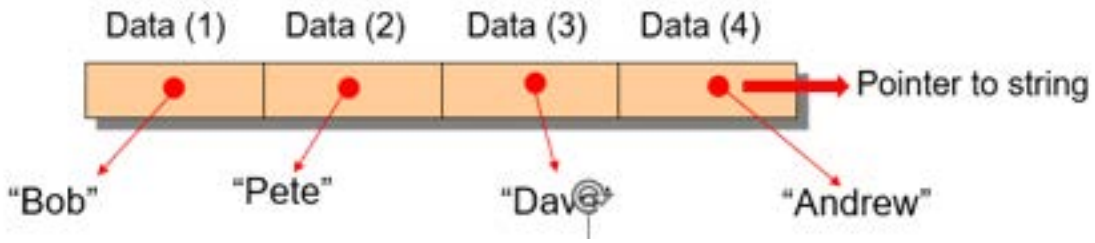
2. Modify the stack package from the Hierarchical libraries exercise to include a generic child procedure. The procedure should iterate through the stack deleting certain elements. The generic parameter should be a function, with an in parameter of type integer, which returns a Boolean indicating whether the element should be deleted from the stack.

Ada83 Access Types

1. Write a main procedure to allow a user to enter names at the terminal. The names entered should then be stored in a list (pointers to strings).

   - Create the types for the above array structure
   - Create an instance (variable/object) of the List
   - Loop through the list (4 times) ask user to enter name (as String)
   - Add the name to the list (pointer to a string)

Once all the names have been added, Loop through the list, dereference the pointer and print each name



Notes:

A pointer to a string should be used to point to the Name. The following variables and procedure call should be used to read a variable length string from the keyboard:

```
InputName    : String (1..80);   -- To hold the string entered
Length       : integer;          -- Holds count of characters entered

begin

  Text_IO.Get_line (Entered_Name , Length);

   -- Now add the name to the list - InputName (1..Length)
```

Ada95 Access Types

1. Declare a record type to hold 1553 message details (RT, SA and Word Count). Declare a number of constants (of the record type), which represent transactions.

   Declare an unconstrained array type whose elements can point to the statically declared transactions (these represent Frames).

   Declare a number constants of the unconstrained type (say, Frame_1,2,3) which contain a list of transactions that would be scheduled for each frame (Make each frame a different length).

   Write an internal procedure (Print_Frame) that takes a Frame as a parameter and prints the details of each transaction.

   Call the Print operation for each frame.

2. Write a main program that contains 5 different internal procedures with the same signature (i.e. each procedure prints a different message to the screen).

   Define a type which represents softkeys (1..5) and create a list (indexed by the softkey number) whose elements point to the internal procedures.

   The user should be able to select a softkey number from the terminal. When selected, the list should be indexed and the attached procedure called.

3. Write a main procedure that contains a type to represent aircraft, the aircraft type should contains two components, the aircraft's tail number and its engine details. The engine details should contain 3 pieces of information, the number of engines connected, their serial numbers (as a natural) and a reference back to the aircraft to which they are all connected.

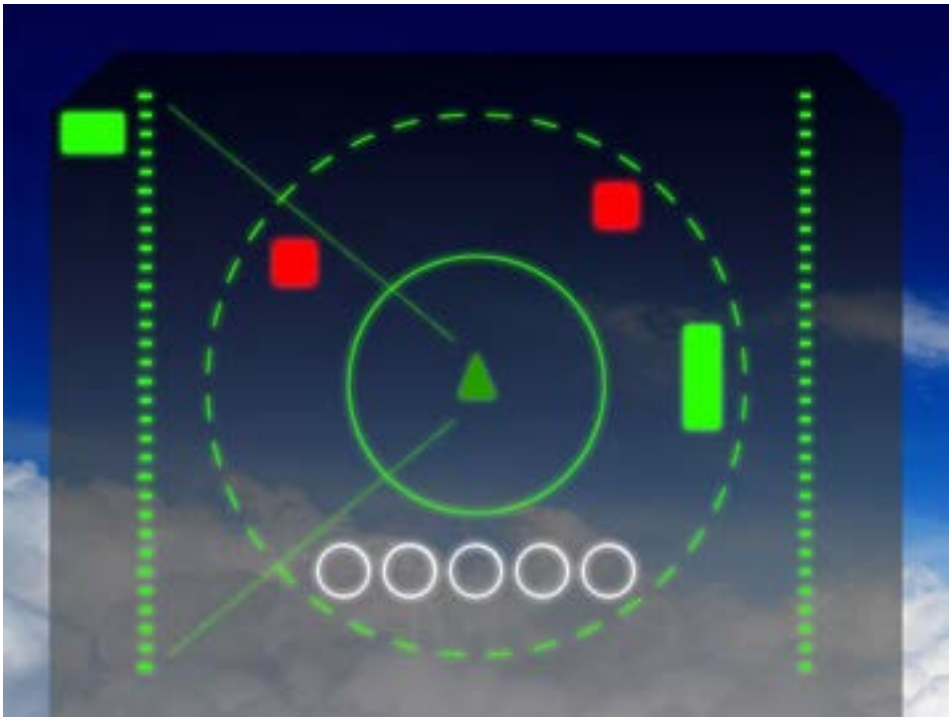
   Declare a Jumbo which has four engines attached. Set-up the tail number and all the engine serial numbers.

   Now print all the information to the screen (including the tail number) by accessing the engine details only, i.e. Find the Tail Number through the engine details.



Hint: The type that represents the engine details will need two discriminant parameters (No_of_Engines & Aircraft_Type)

Object-Oriented Programming



Create an implementation of the above UML in Ada95. Identify which operation is being called by using Ada.Text_IO. This is to be achieved in **4 steps**:

1. Create a class that represents the symbols that will be displayed on the screen. It should have a draw operation which shall output (using Text_IO) its location i.e., its Row and Column. Remember to create methods to allow the objects attributes to be set. Create an instance of Symbol in a main test program and test that its position can be displayed. **Tip: Use the "access" and "aliased" keywords when declaring the object.**

2. Extend the Symbol class with a class called Circle. The Circle class should have an additional private attribute called Radius and also a Constructor to set up its Radius (don't forget its parent's Row and Column). Override the Draw operation so that it outputs its Radius as well as Row and Column. Make an instance of the Circle class in the main test program and test that the Radius, Row and Column are correctly output. **Tip: Use the "tagged" keyword in the Figure class and also re-use the Figure's Construct and Draw behaviour from within the Circle.**

3. Extend the Figure class with a class called Rectangle. The Rectangle class should have two additional private attributes called Height and Width. Override the Draw operation so that it outputs its Height and Width and its Row and Column. Override The Figure's Area operation in Rectangle so that it actually returns the area of the Rectangle. Make an instance of the Rectangle class in the main test program and test that the Height, Width, Row and Column are correctly output. Also, call the Area operation and output it.

4. Make Symbol and the Area operation <<abstract>>. Implement Area in Circle, call it from the main test program and output it. Create an array of symbols in the main test program. The array elements should be an access type to the Symbol class. Fill the array with references to the Circle and Rectangle objects previously created. Loop through the array calling the Area operation on each element of the array. Does the appropriate behaviour get invoked? What is happening here? Extend the Rectangle class with a Square class. Introduce another element to the set of symbols and insert a reference to an instance of the Square class. Test iterating through the list again, is the appropriate behaviour exhibited? **Tip: Use the "abstract" keyword in symbol and create a class-wide ('class) access type to Symbol in the main test program**.

Controlled Types

1. Write a package “Statistics” that contains one type and the procedure “Display_Object_Information”. You should ensure that users cannot copy objects of the type. The procedure is required to display the following information:

    - The current number of objects of the type that exist.
    - The maximum number of objects that have existed.
    - The length of time the longest object lived for.



Note: A Limited Private controlled type should be used.

The main program should declare variables, delay etc. to check the package works correctly.

Ada83 Exceptions

1. Write a main program that contains an internal function **Sum** that takes two parameters of an unconstrained array type. The array type should be an unconstrained array of integer values.

   The function should raise the exception "**Size_Error**" if the arrays are of different sizes and "**Bounded_Error**" if the array bounds are different. Otherwise the elements of the arrays should be added together and the result returned.

   The main program should contain declare arrays of varying sizes and bounds, the arrays should be passed to the function Sum an exception handler used to prints the type of exception that is raised.

2. Modify the Stack package to raise an underflow exception if the stack is empty and an attempt is made to Pop an item and an overflow exception is raised if the stack is full and an attempt is made to push another item.

Ada95 Exceptions

1. Modify the exception handler, from Ada83 Exceptions, exercise 1, to use the **Ada.Exceptions** package to print the name and any information about the exceptions raised (Size_Error / Bounded_Error).

2. Modify the above program to use only one exception **Sum_Error** but ensure the exception handler can determine the type of exception raised (Size or Bounded).



Hint: Use Ada.Exceptions.Raise_Exception and attach the appropriate message.

Ada83 Tasking

1. Write a procedure that contains a task, which writes the word “Iteration” to the screen as parameterised by entry parameters. The task should contain one entry point “Execute” with the parameters that specify:

    a) The number of times the string should be printed
    b) The delay between the print operations.

The entry should not return control back to the caller until the printing is complete.



Hint: Place a line of text_io directly after the entry call.

2. Modify the above program so that the rendezvous is complete before any printing is performed. Test that the main program continues in parallel to the task.



Hint: Place a line of text_io directly after the entry call.

3. Modify the above program so that “Wake Up” is printed if no iterations of the task are requested within 10 seconds.



Hint: A select statement will be required.

Ada95 Tasking

1. Write a procedure that contains a task with a discriminant. The discriminant should be an access to a procedure which takes a single parameter of type string. Declare two local procedures that match the signature of the discriminant, one that prints the string forwards and the other backwards. The discriminant should have a default which points to operation that prints the string forwards.

   The task should contain an entry point with three parameters, the string to print, the number of times to print it and the delay between printing. To print the string, the discriminant should be used. The task timing should use the delay until construct.

   The main procedure should declare two tasks, one that prints forwards, the other in reverse. Check each task executes as expected.

2. Write a procedure that contains a protected type that holds an aircraft height and speed. The protected type should allow the height and speed to be "Updated". The protected type should also provide the facility to "Read" the aircraft details, but **not** until they have been written to at least once.

   Test that calling the Read operation before the Update operation causes the program to be suspended.

Interfacing

1. Assume a message arrives from a missile as a sixteen bit word representing its status as follows:

Bits (0 .. 5) =	Mode : Searching => 09 Hex Locked_On => 12 Hex Failure => 24 Hex Standby => 2A Hex
Bits (6 .. 12) =	Range_To_Target (0 to 127 miles)
Bits (13 .. 15) =	Identity (0..7)

Write a program that accepts user input (to simulate the 16 bit word) and converts it to the appropriate typed representation. Print the converted values to the screen.

Ensure that the program does not try to print any out of range values, but does inform the user that the value is out of range. The program should exit when the user enters a value of zero.

Variant Record Types

1. Declare a variant record to hold aircraft details for 3 different types of aircraft (glider, passenger, and military). The details should include:

All	=> Airspeed
Passenger	=> Fuel load Number of passengers Number of vegetarian meals
Military	=> Fuel_load Number of bombs

Declare an object of each aircraft type and assign each object a set of default values.

Declare an array of 3 aircraft, one of each type and write a procedure to print the aircraft details. Iterate through the list calling the display procedure for each element of the array.

Unconstrained Array Types

1. Assuming a system contained 20 softkeys, create a structure capable of holding the details for each key. The details should include:

    - Softkey legend (string 1.. 5)
    - Legend Position (Row, Column).

Create a constant to hold details for only 2 softkeys and populate it with example details. Iterate through the array and print all the details for each key on the display.



Hint. An unconstrained array of records should be used.

Discriminated Record Types

1. Write a program that contains a discriminated record containing softkey details. The softkey details should include:

    - Row
    - Column
    - A variable length legend (String up to 5 characters)

Declare two objects of the discriminated record type, one with a legend of "**ADC**" and one with a legend of "**TIALD**".

Write an internal procedure that takes in the discriminated record, as a parameter, and displays all the discriminated record details on the screen.

2. Write a procedure that can transpose squares of size 1X1 to 10X10. Declare two internal operations, Transpose and Print. The transpose operation transposes the square and the print operation displays the contents on the screen. Declare two squares of different sizes, display their initial contents, call the internal operation to transpose them and then print their new values.

Ada95 Predefined Library

1. Using the Ada.Numerics.Discrete_Random package, write a short program to deal cards when requested to by the user. Do not worry about remembering whether a card has been dealt before**.**



Hint: You will need two instantiations (one for the suite and one for the card)

2. Write a program that contains an unbounded string. Set the string to “EFA was developed by British Aerospace and first flew in 1994” and find the following:

   - The number of capital letters (using a character set).
   - The position that British Aerospace appears within the string.
   - Replace “British Aerospace” with “BAE Systems.

Ada 2005 – Interfaces

1. Assume you have to provide an interface to a cockpit terminal that contains a screen and 20 keys. Define 2 interfaces:

   - iScreen Operations to SetRow, SetColumn and Print
   - iKey An function to return the last key number pressed

Now realise the interface in a class called MPCD that includes an attribute to indicate whether the aircraft is in the air or on the ground and operations to Set and Get the current In-Air state.



Note: SetRow, SetColumn and Print can use Text_IO to indicate they have been called.

Write a main program that tests the MPCD class.

Use the reserve word **overriding** to indicate which operations are being overridden and not **overriding** for the others. Try **overriding** a non-inherited operation such as SetInAir!

Ada 2005 – Access Types

1. Without the use of any named access types or forward declarations, create a FIFO class (implemented as a linked list) that provides the following functionality:

IsEmpty	- Function that returns Boolean (True if FIFO is empty)
Count	- Returns the number of elements in the FIFO
Add	- Operation to add (Integer value) to the end of the list
Get	-Returns the next value in the list (First in First out)



Note: Do not worry about memory leaks!

Ada 2005 – Synchronized Interface

1. Create an Integer FIFO interface class that provides the operations (as in previous exercise):

    - IsEmpty - Function that returns Boolean (True if FIFO is empty)
    - Count - Returns the number of elements in the FIFO
    - Add - Operation to add (Integer value) to the end of the list
    - Get - Returns the next value in the list (First in First out)

The interface should be synchronized to ensure it is implemented in a manner that is thread safe, i.e. no one call interrupt the processing during the Add or Get operations.

Create a realisation of the interface and implement the synchronisation using a protected type.

Test your realisation works.



Notes:

- **Do not worry about memory leaks!**
- **You could add a private entry to implement the Get**
- **The Get could be guarded (only allow when count > 0)**
- **Use overriding & not overriding**

Time permitting, modify your Integer FIFO and create a generic FIFO that can be instantiated to store any type. Instantiate the FIFO with various types (Integer, Boolean) and then implement the interface realisations.

Systems Programming Annex

1. Write a program that prints an asterisk '*' on the screen every second. Implement a static interrupt handler that prints the message 'CTRL-C Pressed' every time the Control-C key is pressed.

   Note: Ada.Interrupts.Names contains a constant called 'SIGINT' that identifies the physical interrupt that is generated whenever CTRL-C is pressed.

2. Write a program to attach an interrupt handler, to the CTRL-C interrupt, that prints 'CTRL-C (1)' on the screen every time the Control-C key is pressed. The program should then print '1' on the screen at 0.5 second intervals for 10 seconds.

   After 10 seconds has elapsed, the program should attach a different interrupt handler, to the CTRL-C interrupt, that prints 'CTRL-C (2)'. The program should then print '2' on the screen at 0.5 second intervals for 10 seconds.

   This behaviour should loop forever.

3. Write a program that contains an enumeration type (Day_Type) representing the days of the week. Loop through the enumeration type printing out the value of each enumerate.

   Note: you can use Day_Type'Image (loop_variable) to get the string representation of each enumerate.

Now use pragma Discard_Names to reduce the amount of storage used for storing names associated with Day_Type and rerun your program

4. Write a program that contains a task (Port_Server) with two entry points, Open and Close. When Open is called, the ID of the calling tasks must be saved. When Close is called, the Port_Server shall determine whether the same task that opened the port is closing it. If not an error message should be displayed, otherwise a successful message shall be displayed.

   To test the Port_Server, create 4 client tasks with entry points Start and Stop. Start should call Port_Server.Open and Stop should call Port_Server.Close.

   Note: Use a task type and create an array of 4 instances.

5. Include a loop in the main program to ask the user what operation should be performed (Start / Stop) and on which Client (1..4). The appropriate client can then be scheduled to test the Port_Server works as expected.

Real-Time Systems Annex

1. Write a program that contains a task type which has one entry point called Start. The entry should take two parameters, an iteration rate and a number representing the tasks identity. The body of the task should wait on the rendezvous to be started. Once started, the task should iterate at the requested rate and should print the task number and its current priority on the screen. The task type should be given a default priority of 10.

   The main program should declare two instances of the task type and start them with different rates and identities.

   The main program should now ask the user to enter a task number (1..2) and a priority. The main program should now change the priority of the entered task using dynamic priorities. The result should be that the new priority of the specified task can be seen on each iteration.

2. Write a program that enforces the following tasking restrictions:

   - No more than 1 task entry
   - No more than 3 select alternatives
   - Only allows tasks at library level

Declare tasks in your main program to ensure the violations are detected.



Note: The Restrictions must be defined in a package specification.

3. Write a program to display the Tick value, defined in the Ada.Real_Time package, on the screen in seconds, milliseconds, microseconds and nanoseconds.

4. Write a program that contains a loop that iterates at a rate of 250 milliseconds. You should use the function Ada.Real_Time.Milliseconds to implement the delay period.

   Each iteration should use the Ada.Real_Time package to determine the current time using the Clock function. The program should then use the operation Split to determine the number of seconds from the Time. The program should then display Hours, minutes and seconds on the screen. The time should only be redisplayed whenever a second elapses.

Numerics Annex

1. Write a program that performs the following complex number calculations and print the result on the screen using Ada.Float_Text_IO to display the values of the real and imaginary parts:

    - (3-5i) + (6+7i)
    - (3-5i) - (6+7i)
    - (3-5i) * (6+7i)
    - (3-5i) / (6+7i)

2. Use a constant to define the first complex number (3-5i) and the constructor (Set_Re & Set_Im) to setup a variable containing the second complex number (6_7i).

3. Use the functions Re & Im to get the complex number component values

4. Perform the same exercise as above using your own floating point type that has nine digits of precision.

5. Modify the above program to use Complex IO to display the values of the calculations on the screen.

6. Using the Ada.Numerics.Discrete_Random package, write a short program to deal cards when requested to by the user. Do not worry about remembering whether a card has been dealt before.



Hint: You will need two instantiations (one for the suite and one for the card).

Safety & Security Annex

1. Write a program that restricts the use of protected types but attempt to declare one. Compile the program to ensure the violation is detected.